Sri Lanka Institute of Information Technology



IT1140 - Fundamentals of Computing
Year 1, Semester 1

FreshGuard: IoT-Based Smart Fruit Bowl

Date of Submission: September 10, 2026

Progress Report

Group 13

IT Number	Name
IT26102355	KULATHUNGA M.A.S.S. (Team Leader)
IT26102361	VINOJA H.W.O.
IT26102357	SUBHASHANA T.O.
IT26102368	PERERA M.K.J.C.
IT26102317	SITHUMINI W.T.I.
IT26102348	DAMAYANTHA H.A.E.

Introduction

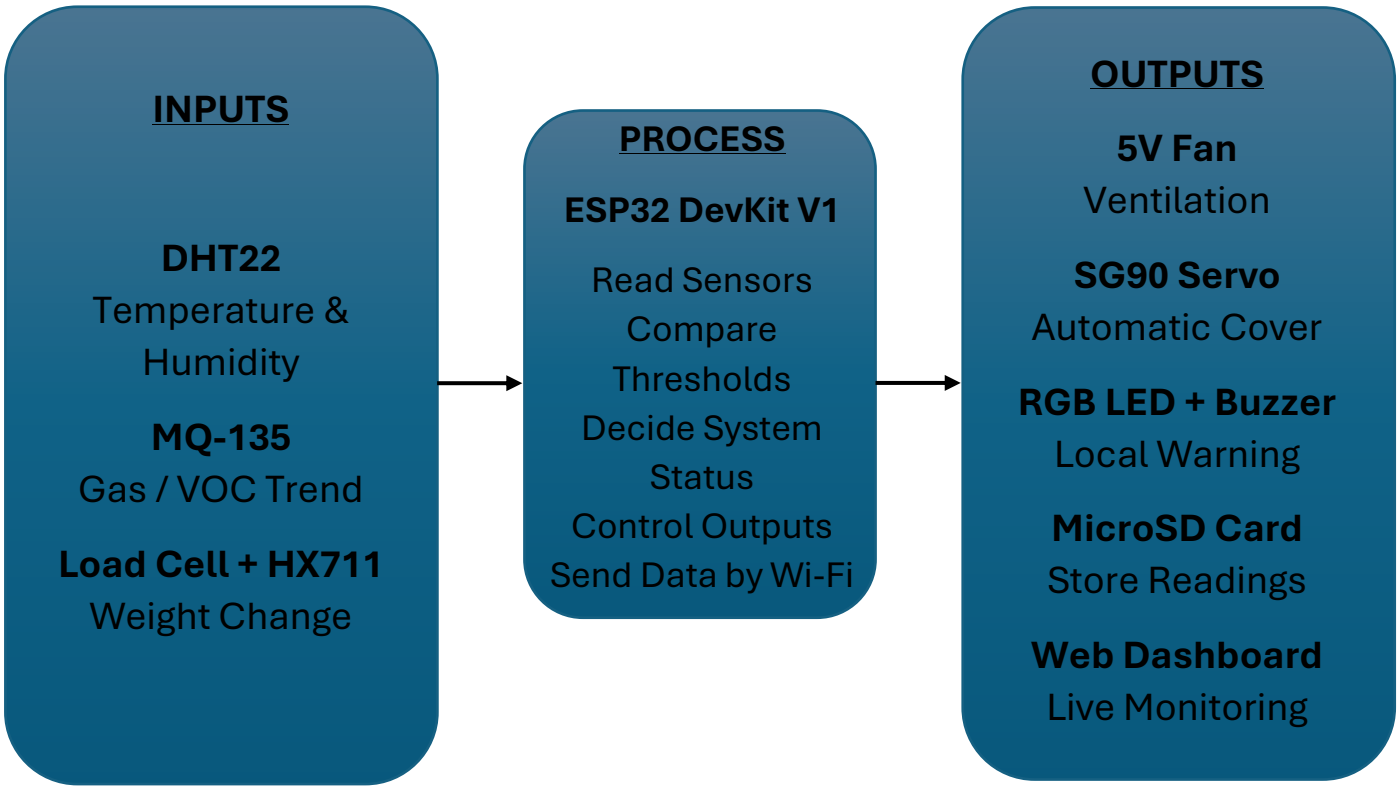
This progress report is submitted as part of the IT1140 - Fundamentals of Computing module during our first year, first semester at the Sri Lanka Institute of Information Technology (SLIIT). Our team, Group 13, is developing a mini-project titled "FreshGuard: IoT-Based Smart Fruit Bowl"

Currently, traditional fruit bowls cannot measure important environmental factors like temperature, humidity, gas level changes, or the weight of the fruits. Because of this, users typically only notice spoilage when fruits become too soft, change color, or produce a bad smell, which leads to unnecessary household food waste.

To address this problem, FreshGuard uses simple sensors and Internet of Things (IoT) technologies to monitor the storage conditions around the fruits. Our system is designed to give users early warnings through local indicators and a Wi-Fi web dashboard, while also automatically activating a fan to improve ventilation when the conditions require attention.

This project gives our group a valuable opportunity to apply the foundational programming and hardware knowledge we are learning this semester to a practical problem. This document outlines our current progress, detailing our proposed system architecture, our work breakdown structure, and our initial proof of work as we build and test the first FreshGuard prototype.

1. System Architecture



The architecture of the FreshGuard system is divided into three main sections: input, processing, and output

Input Section:

The system uses a set of sensors to collect continuous data about the environment around the fruits. A DHT22 sensor measures temperature and humidity, while an MQ-135 sensor monitors gas and VOC trend changes. Additionally, a load cell connected to an HX711 amplifier is used to measure changes in the weight of the fruits.

Processing Section:

The main controller is the ESP32 DevKit V1. The ESP32 receives the data from the input sensors, filters the readings, and compares them against our selected threshold values. Based on these comparisons, the microcontroller classifies the storage condition as "Fresh," "Attention," or "Possible Spoilage" and decides the necessary actions

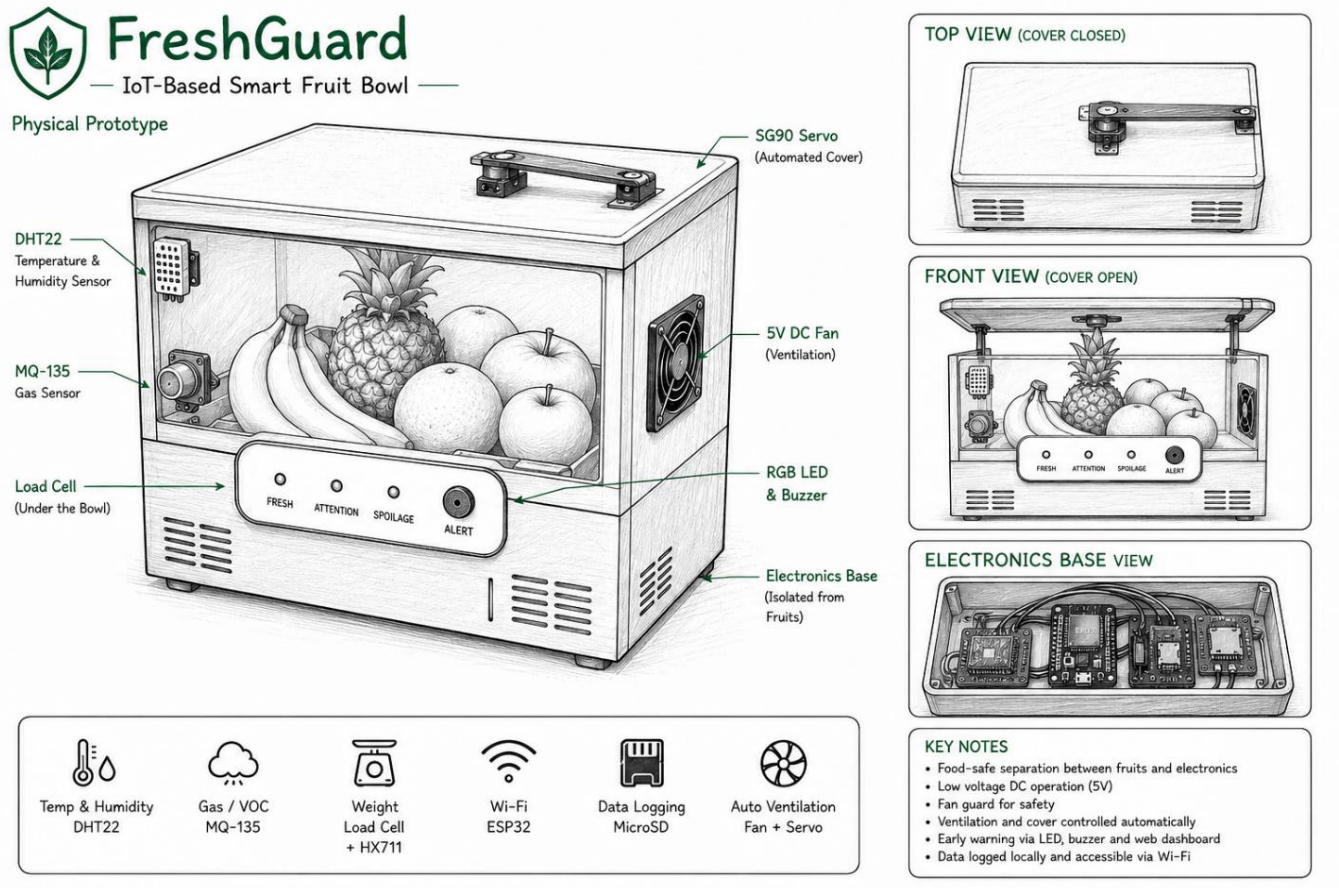
Output Section:

According to the processing results, the ESP32 controls the system outputs. It operates a 5V DC fan to improve ventilation and an SG90 servo motor to control an automatic cover. For local alerts, an RGB LED and an active buzzer are triggered to warn the user. The ESP32 also saves the historical sensor readings into a MicroSD card and sends the live data over Wi-Fi to a web dashboard for easy monitoring.

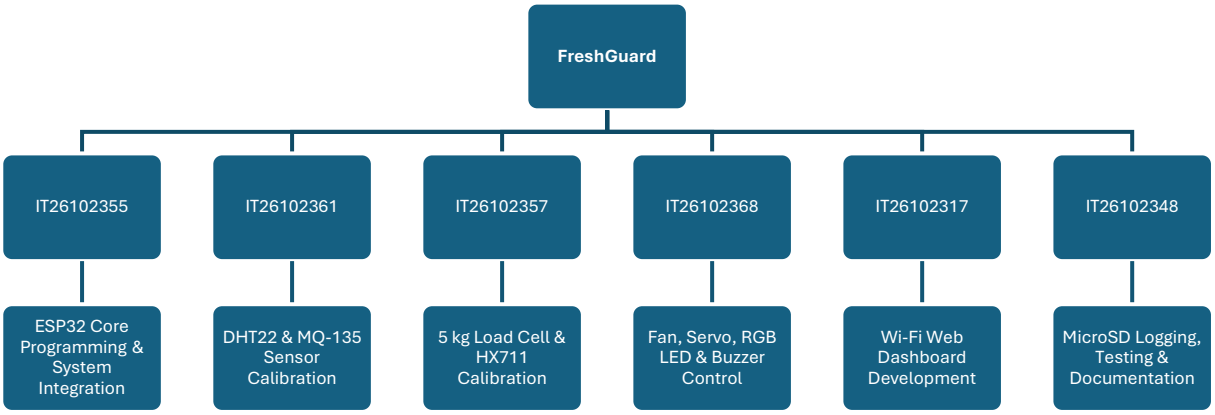
Physical Prototype Layout

To ensure safety and reliability, the physical architecture of the prototype separates the electronic components from the fruit storage area. The sensors measuring air conditions (DHT22 and MQ-135) are placed in the upper section near the circulating air. Meanwhile, the main processing units and power electronics are safely isolated in a lower base to protect them from moisture and mechanical stress.

The outer enclosure and structural base of the initial physical prototype will be constructed using rigiform. This material was chosen to allow for rapid prototyping, easy modification of component placements, and safe housing of the electronics.



Work Breakdown Structure



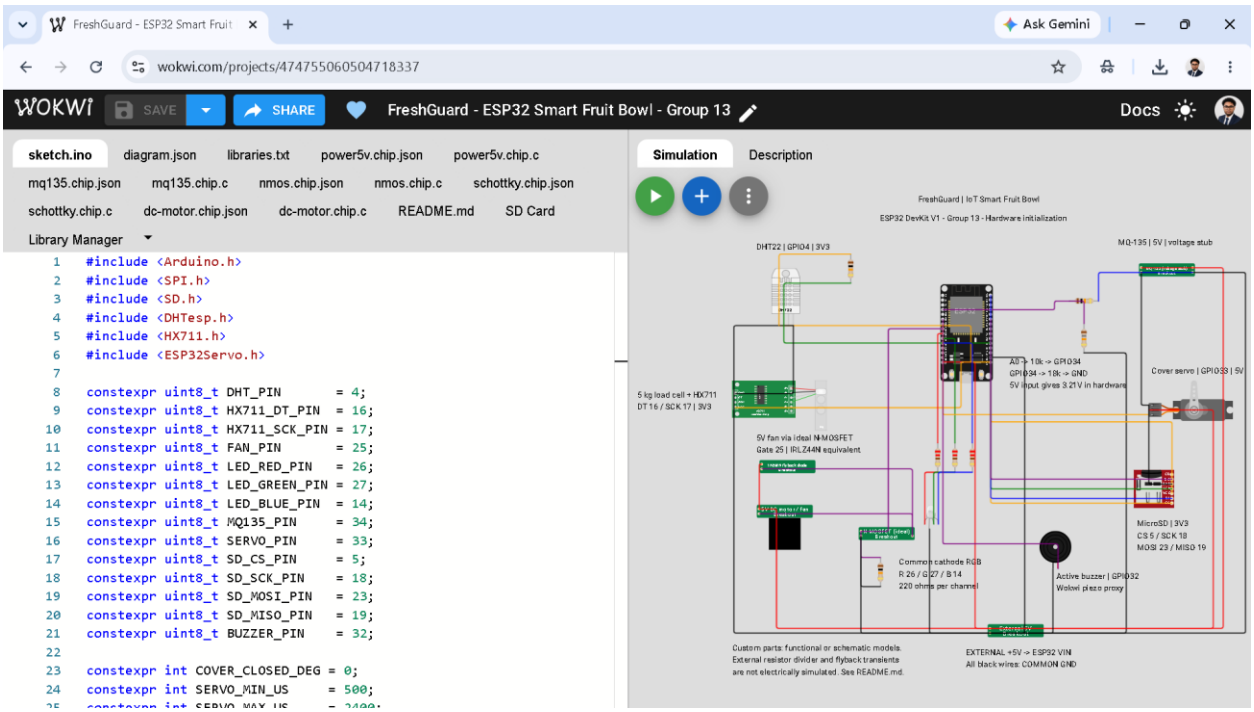
Phase / Task ID	Task Description	Assigned Team Member(s)	Timeline / Milestone
Phase 1	**Research, Proposal & Presentation**		
1.1	Problem definition, literature review & proposal document	Kulathunga M.A.S.S. & Vinoja H.W.O.	Weeks 1 - 2
1.2	System architecture design & proposal presentation	All Team Members	Weeks 2 - 3
Phase 2	**Hardware Procurement & Prototyping**		
2.1	Hardware collection & component inspection	All Team Members	Weeks 4 - 5
2.2	ESP32 setup and sensor integration (DHT22, MQ-135)	Sithumini W.T.I. & Damayantha H.A.E.	Weeks 5 - 6
2.3	Actuator integration (Fan, Servo, Buzzer, RGB LED)	Kulathunga M.A.S.S. & Subhashana T.O.	Weeks 6 - 7
2.4	Weight sensor calibration & MicroSD logging	Vinoja H.W.O. & Perera M.K.J.C.	Weeks 7 - 8

Phase 3	**Software, Testing & Final Integration**		
3.1	Complete hardware testing	Kulathunga M.A.S.S	Weeks 8 - 9
3.2	Wokwi simulation	All Team Members	Weeks 8 - 9
Phase 4	**Documentation**		
4.1	Progress report compilation and formatting	All Team Members	Week 9

Proof of Work

1: Wokwi Circuit Simulation and Hardware Mapping

This image illustrates the comprehensive virtual hardware setup for the FreshGuard system using the Wokwi simulation platform. It demonstrates the complete wiring and pinout configuration between the central ESP32 DevKit V1 microcontroller and all required peripheral components.



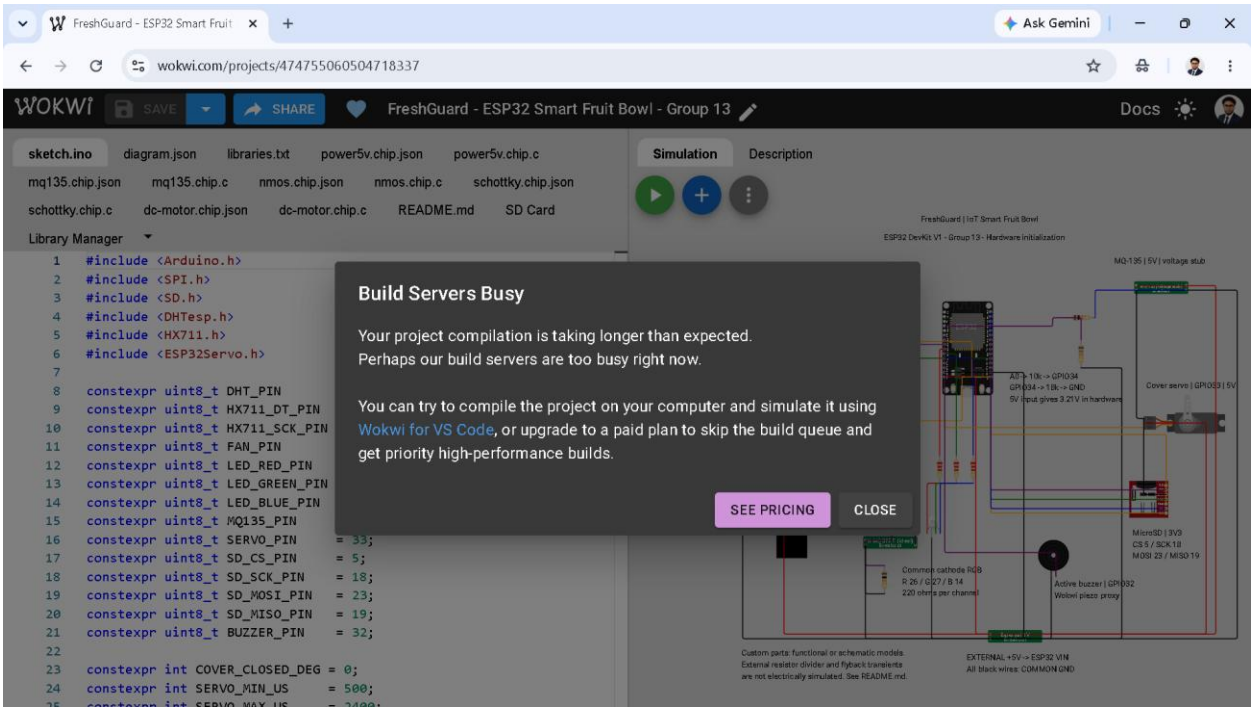
Key integrations shown in the diagram include:

Input Sensors: The DHT22 (temperature and humidity), MQ-135 (air quality/gas), and an HX711 module connected to a 5kg load cell for weight monitoring.

Output Actuators: A 5V DC fan driven by an N-MOSFET for ventilation, a servo motor for the automated cover, a common cathode RGB LED for visual status alerts, and an active buzzer for audio warnings.

Data Logging: A MicroSD card module wired via SPI communication to store historical environmental readings.

We attempted to run the Wokwi simulation at least ten times, but repeated server busy messages prevented us from completing simulation testing. The Wokwi diagram therefore documents our proposed wiring and component connections; its operation has not yet been verified through simulation. We carried out individual component tests using the physical ESP32 hardware, as shown in the following photographs and Arduino Serial Monitor screenshots. Simulation testing remains pending.



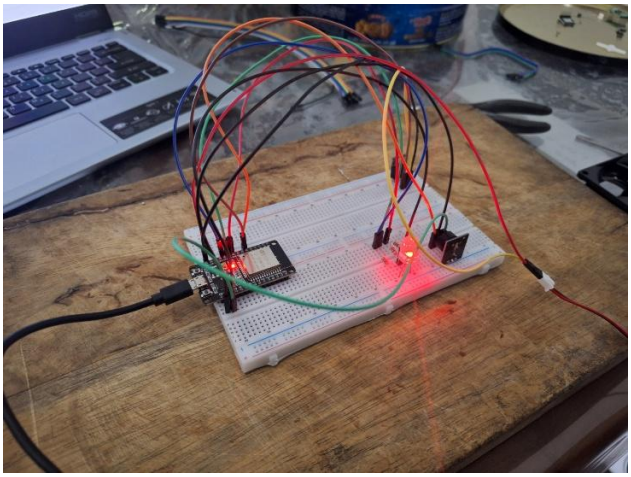
[View the Interactive Wokwi Simulation Here]

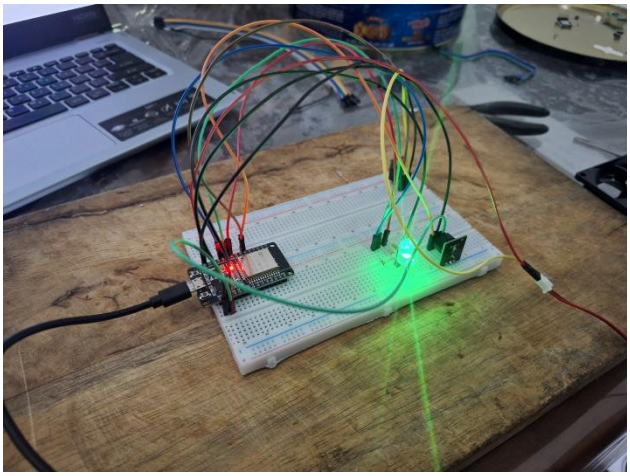
2: Hardware Testing and Prototype Iterations

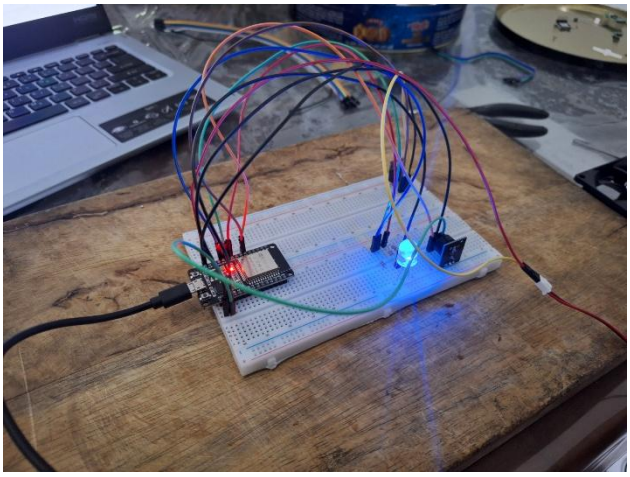
1: RGB LED Output State Verification

Description: Testing individual color channels (Red, Green, and Blue) via the ESP32 microcontroller to ensure correct GPIO pin mapping and output logic

Verification: Confirmed by matching the active LED illumination states with corresponding serial monitor outputs in the Arduino IDE.



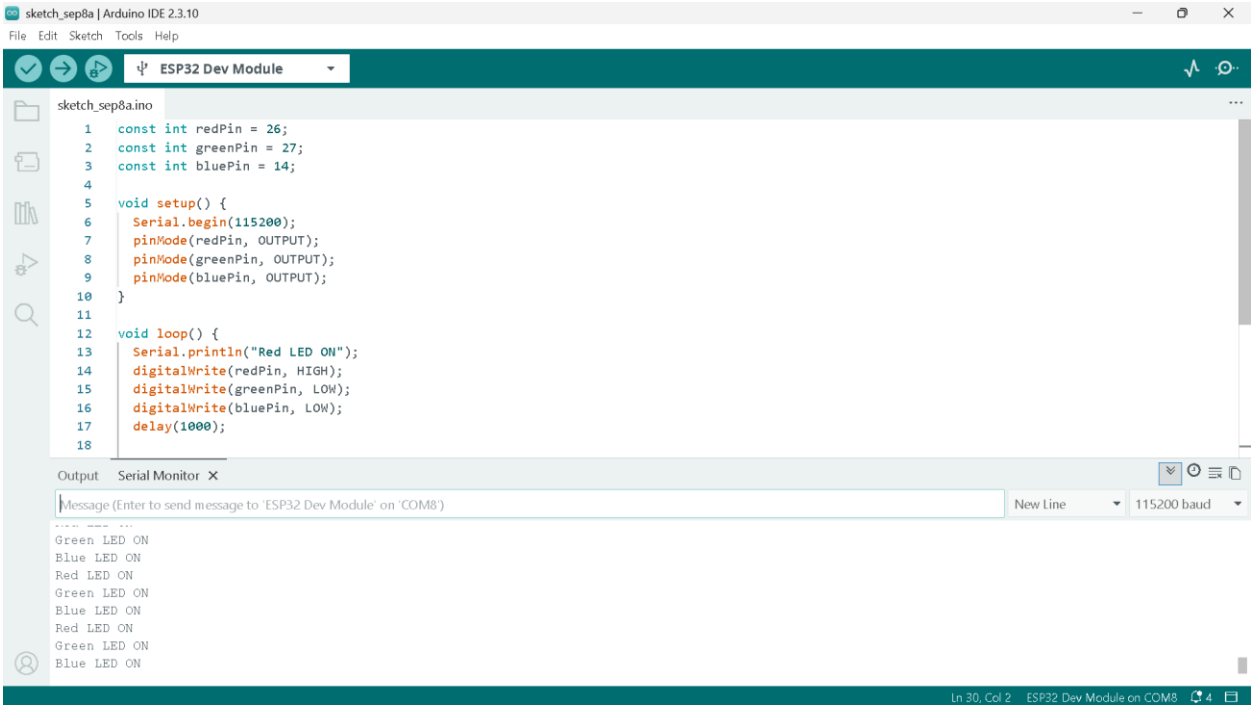




Firmware Implementation and Serial Monitor Verification

Description: Arduino IDE screenshot showing the code configuration for the ESP32 GPIO pins (Red: 26, Green: 27, Blue: 14) alongside the Serial Monitor output confirming correct execution and logic flow.

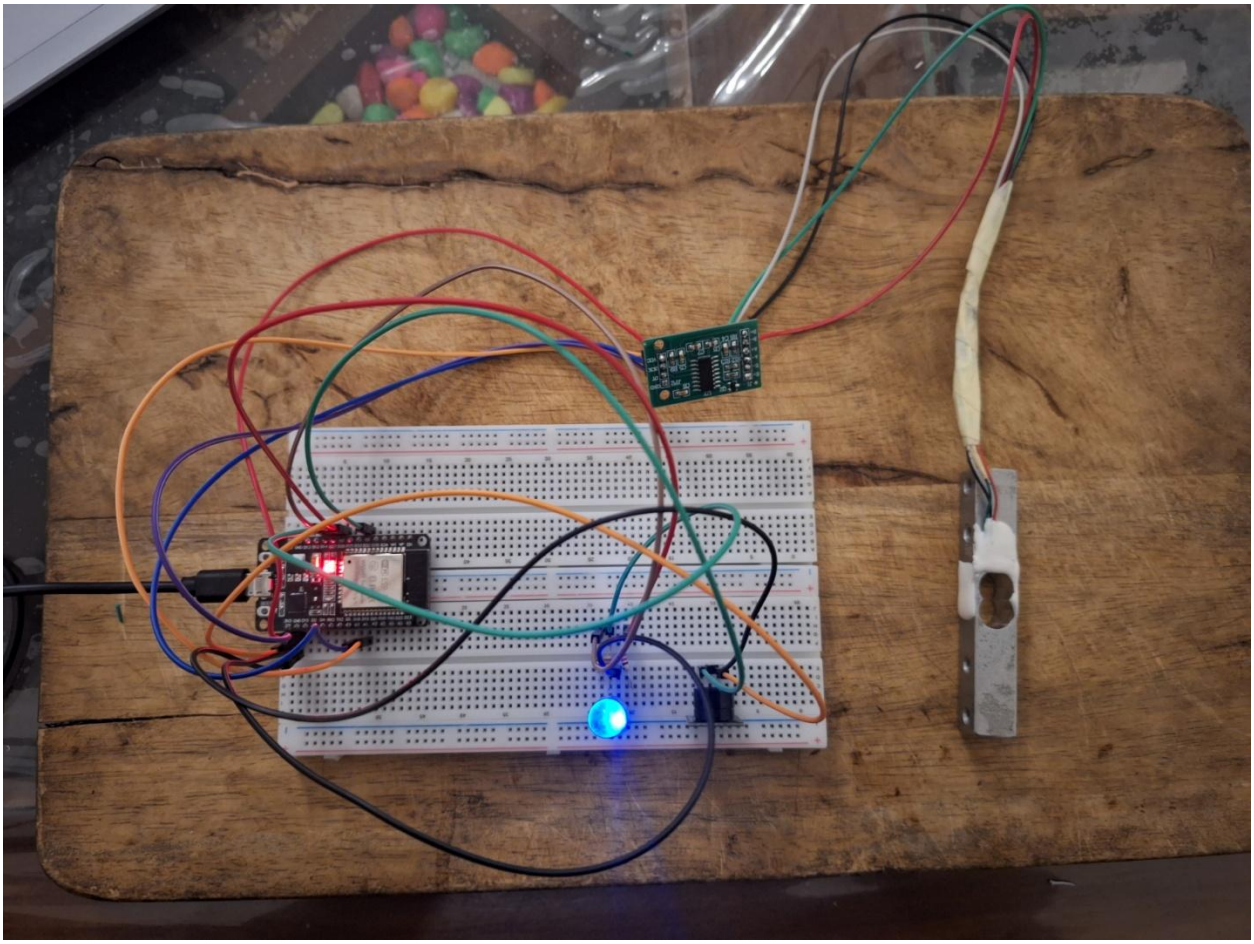
Verification: Confirmed by active serial logging (Red LED ON, Green LED ON, Blue LED ON) matching the physical component behavior.



2: Load Cell Sensor and HX711 Amplifier Integration

Description: Physical circuit setup incorporating an HX711 ADC module connected to a load cell (weight sensor) and the ESP32 to measure weight increments.

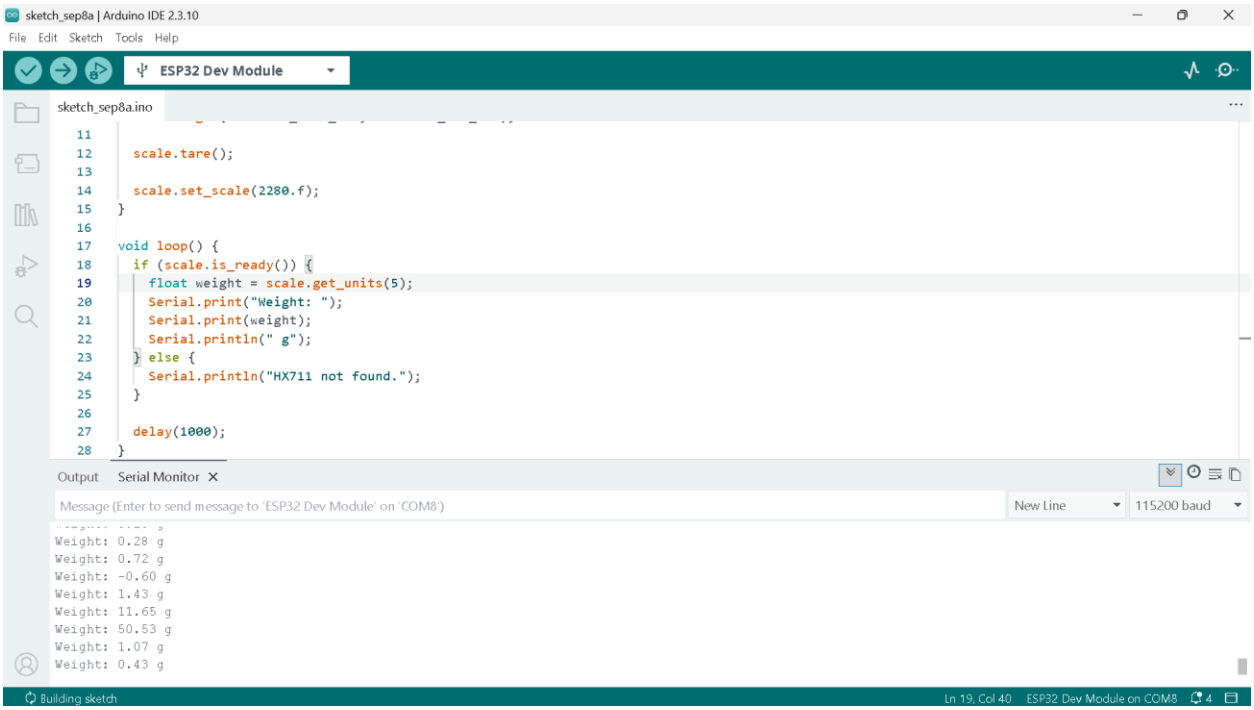
Verification: Confirmed by active weight readings displayed in grams on the Arduino IDE Serial Monitor during testing



HX711 Firmware Code and Weight Data Output

Description: Arduino IDE screenshot showing the sensor initialization script (scale.tare(), scale.set_scale()) and live serial logging output displaying measured weights.

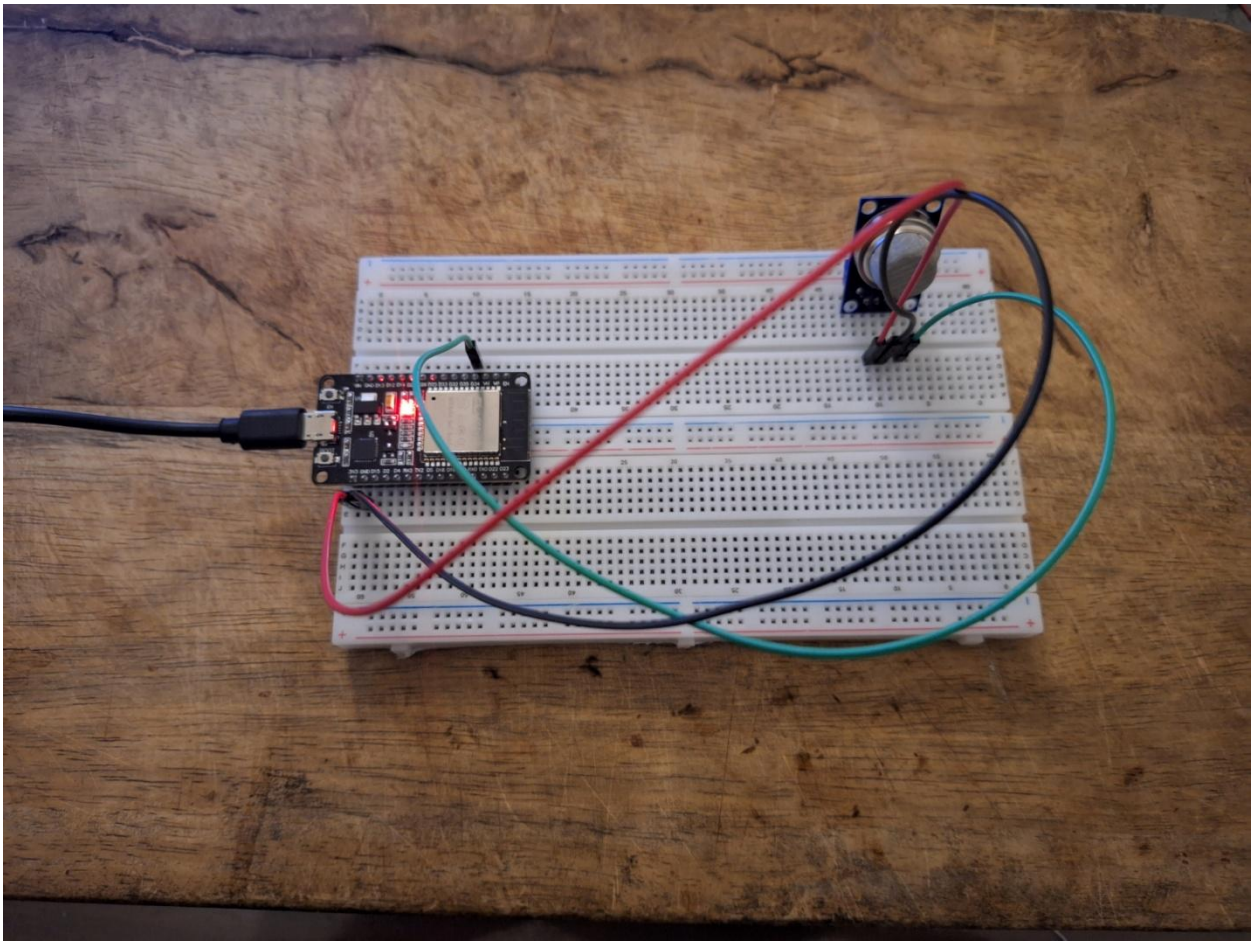
Verification: Verified by continuous stable numerical readings (Weight: X g) appearing in the serial console output



3: MQ-135 Air Quality Sensor Integration

Description: Physical circuit setup wiring the MQ-135 gas and air quality sensor module to the analog input pin (GPIO 34) of the ESP32 development board on the breadboard.

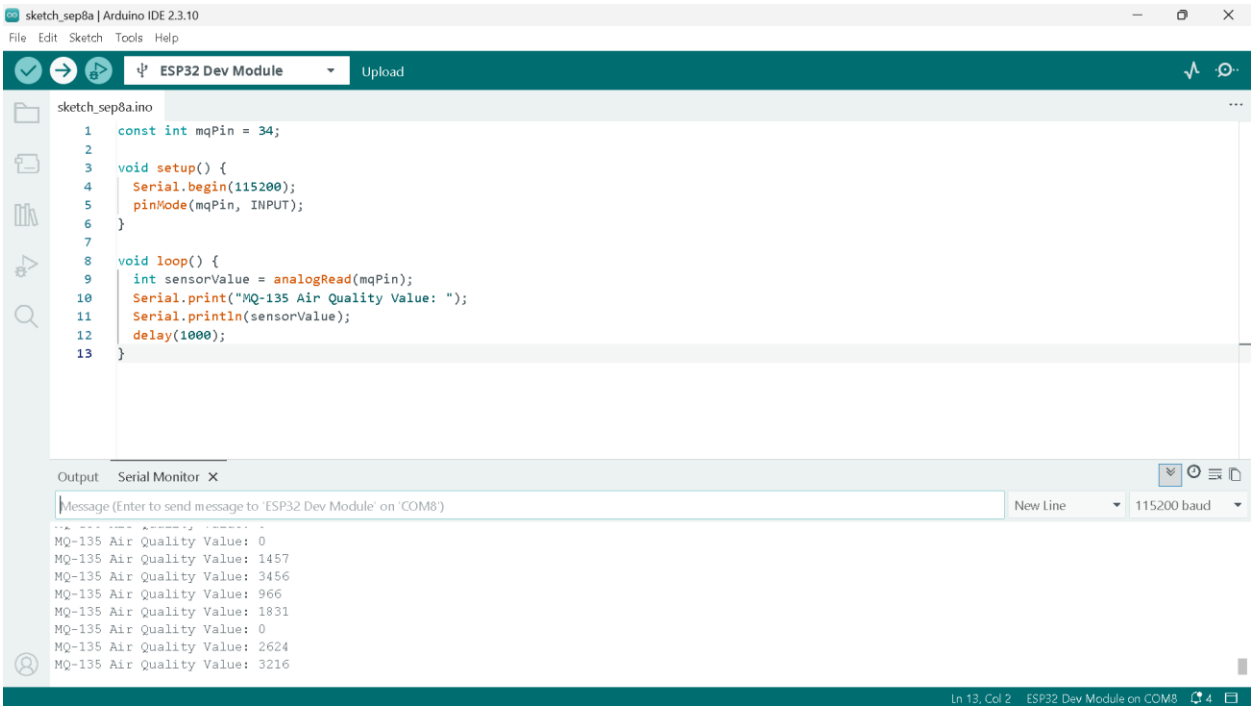
Verification: Confirmed by fluctuating numerical raw sensor values logging successfully in the Arduino IDE serial monitor



Air Quality Monitoring Firmware and Serial Output

Description: Arduino IDE code snippet reading analog inputs from mqPin = 34 and outputting real-time air quality values (MQ-135 Air Quality Value: [value]).

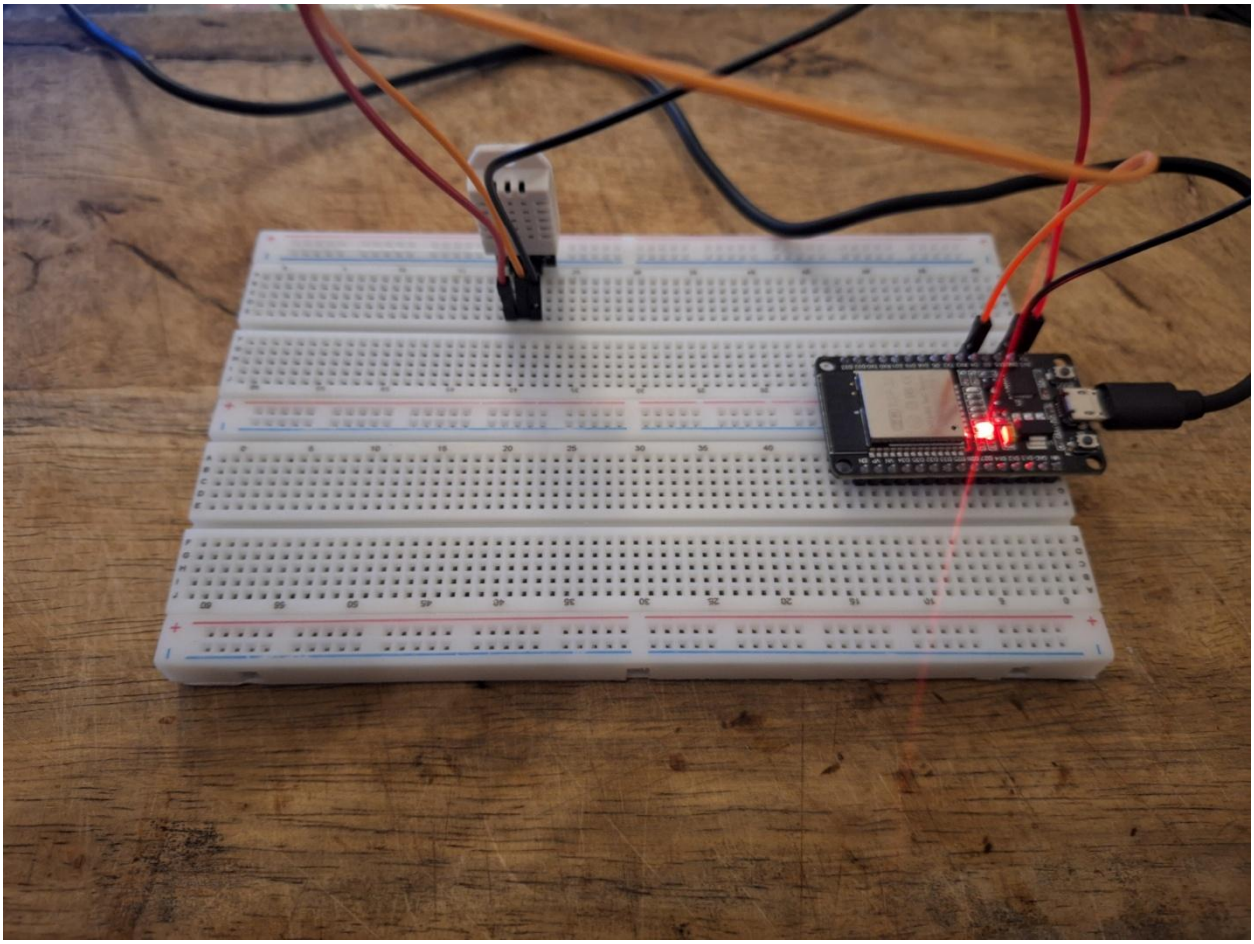
Verification: Verified by active real-time data streaming in the serial console output displaying varying environmental readings.



4: DHT22 Temperature and Humidity Sensor Integration

Description: Physical breadboard assembly connecting the DHT22 environmental sensor module to the ESP32 digital pin (GPIO 4)

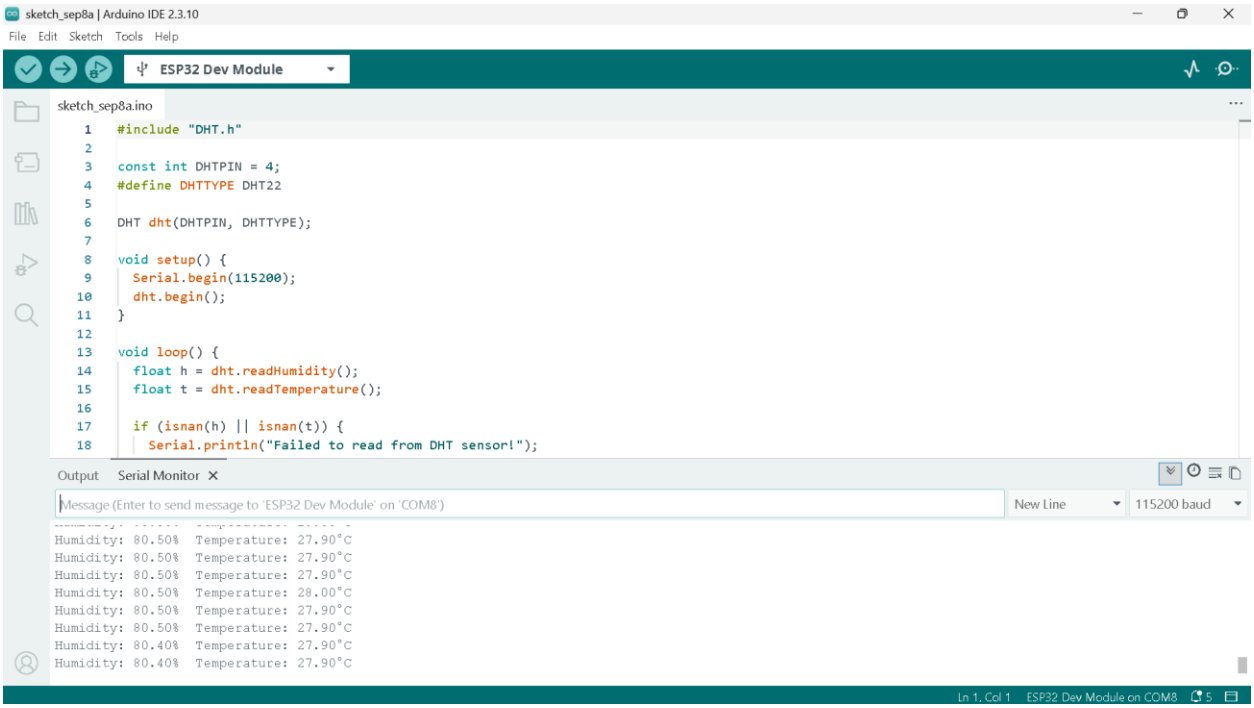
Verification: Confirmed by continuous real-time logging of accurate ambient environmental metrics (Humidity: 80.50% Temperature: 27.90°C) in the serial console output.



DHT22 Sensor Firmware and Environment Monitoring Output

Description: Arduino IDE code snippet using the DHT.h library to poll temperature and humidity readings from the sensor via dht.readTemperature() and dht.readHumidity().

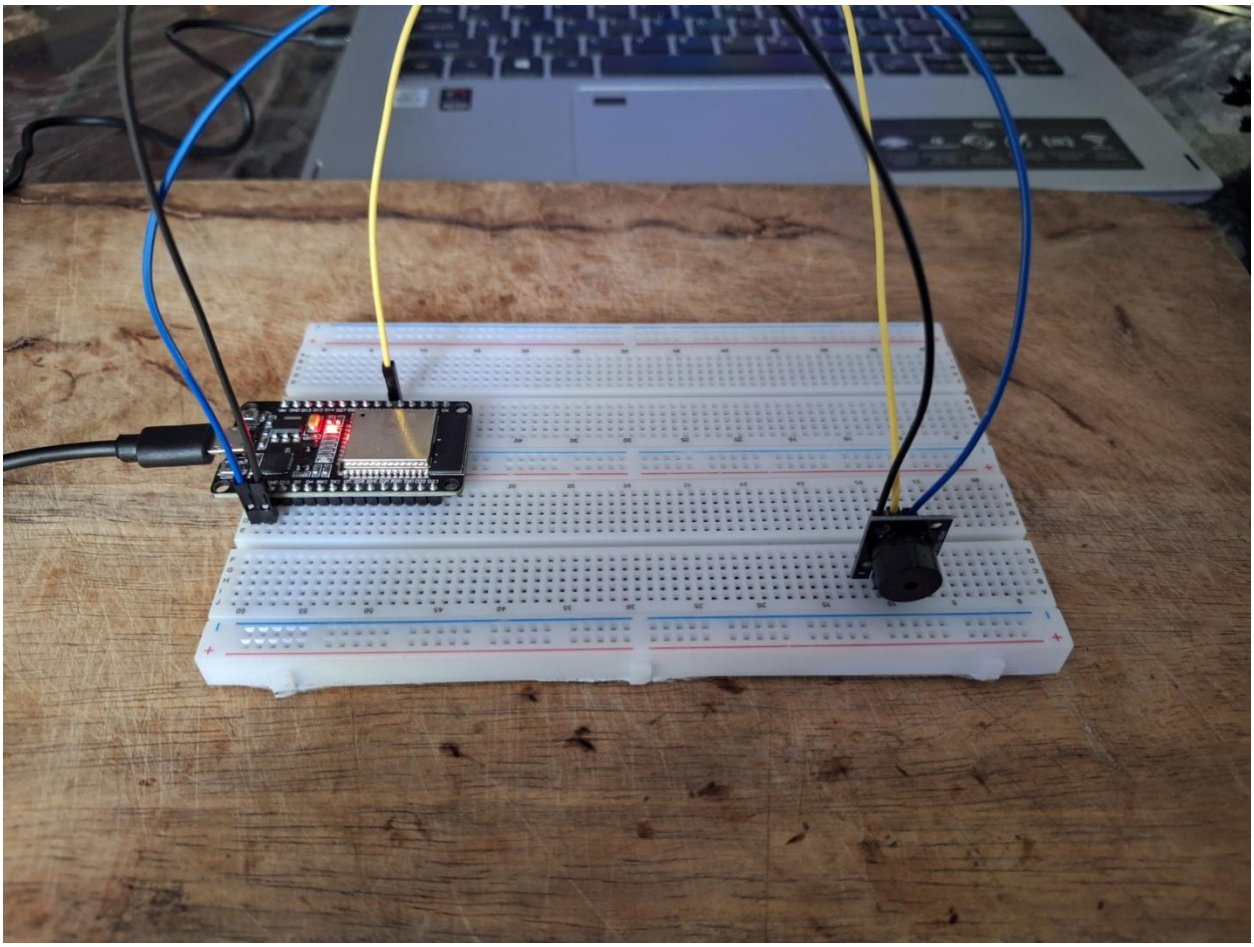
Verification: Verified by stable, updating environmental values streaming into the serial monitor without error messages.



5: Piezo Buzzer Hardware Integration

Description: Physical breadboard wiring connecting a digital piezo buzzer module to the ESP32 microcontroller development board (GPIO 32).

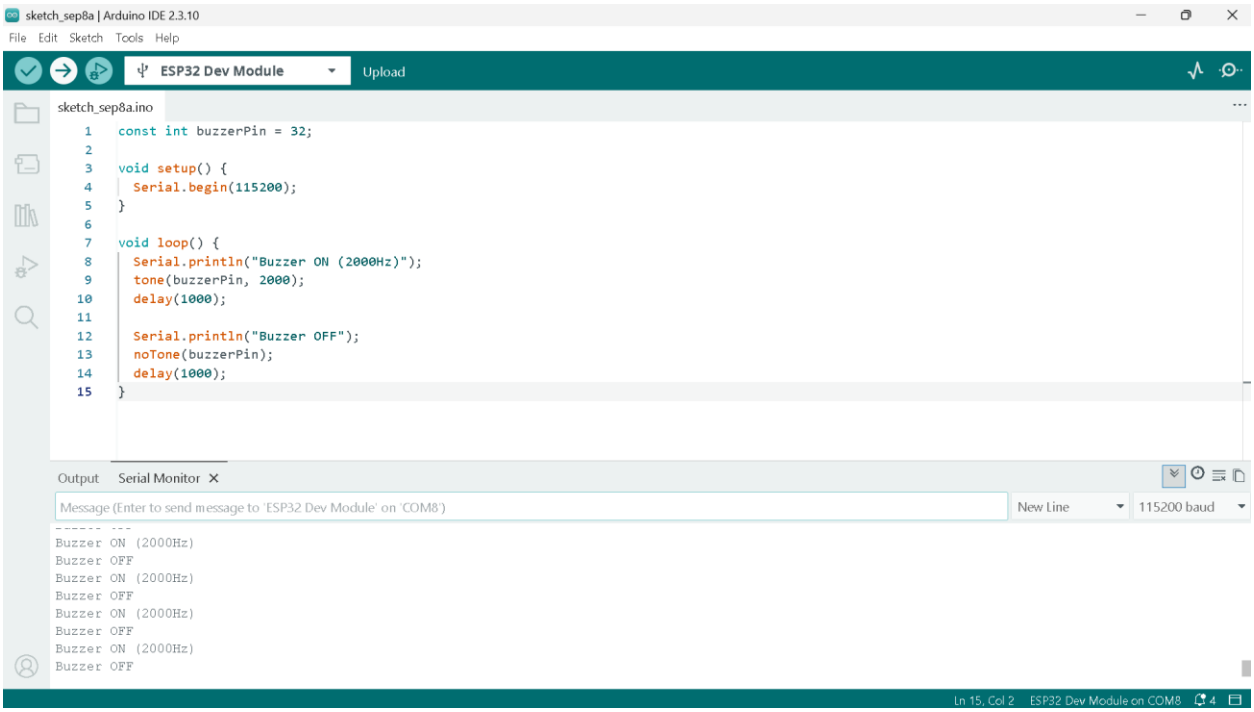
Verification: Confirmed by audible tone output matching cyclic timing sequences during testing.



Piezo Buzzer Control Firmware and Serial Output

Description: Arduino IDE code snippet utilizing the tone() and noTone() functions on buzzerPin = 32 to generate acoustic alerts alongside status logging.

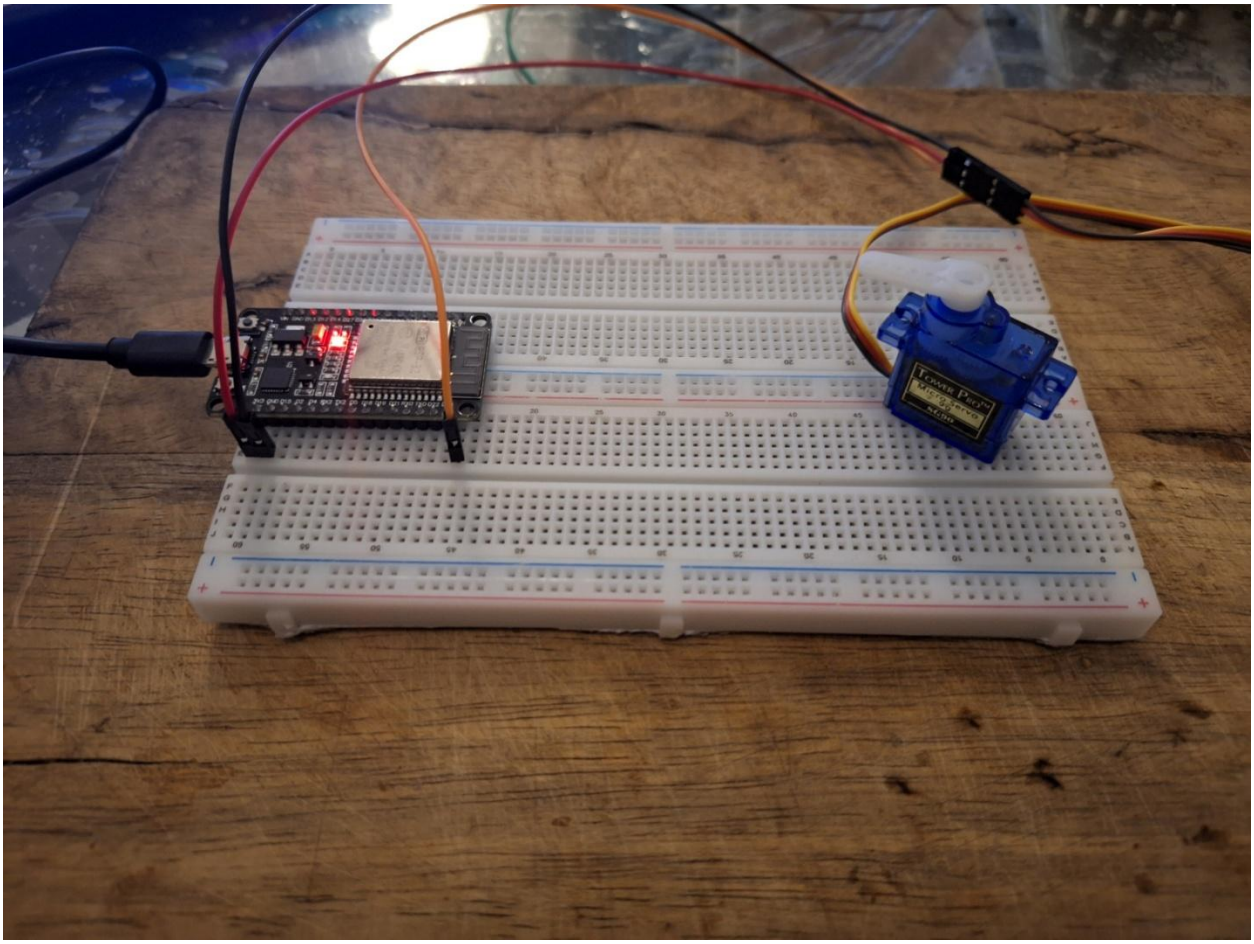
Verification: Verified by corresponding console logs (Buzzer ON (2000Hz) and Buzzer OFF) syncing with the physical audio output.



6: Micro Servo **Motor Hardware Integration**

Description: Physical breadboard and jumper wire connections wiring a TowerPro micro servo motor to the ESP32 development board for rotational control.

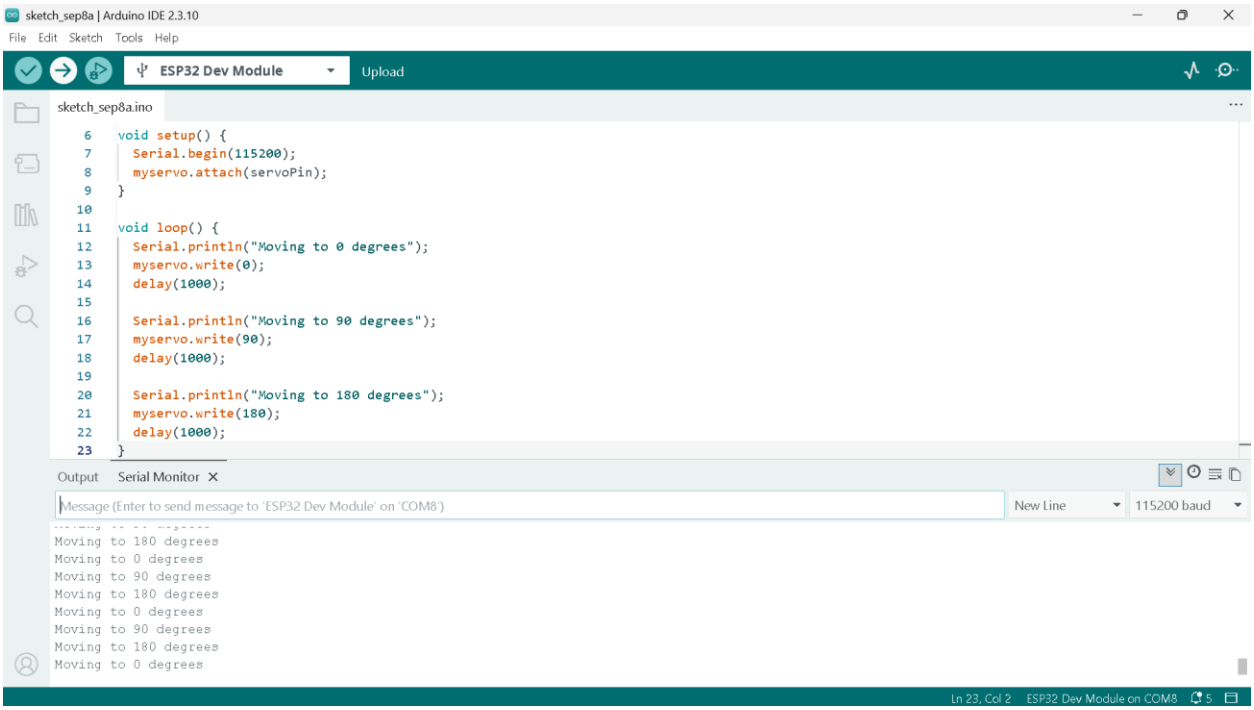
Verification: Confirmed by smooth mechanical rotation and sweeping arm movement during firmware execution.



Servo Motor Control Firmware and Serial Output

Description: Arduino IDE code snippet using the Servo library to sweep the motor position across specific angles (0, 90, and 180 degrees).

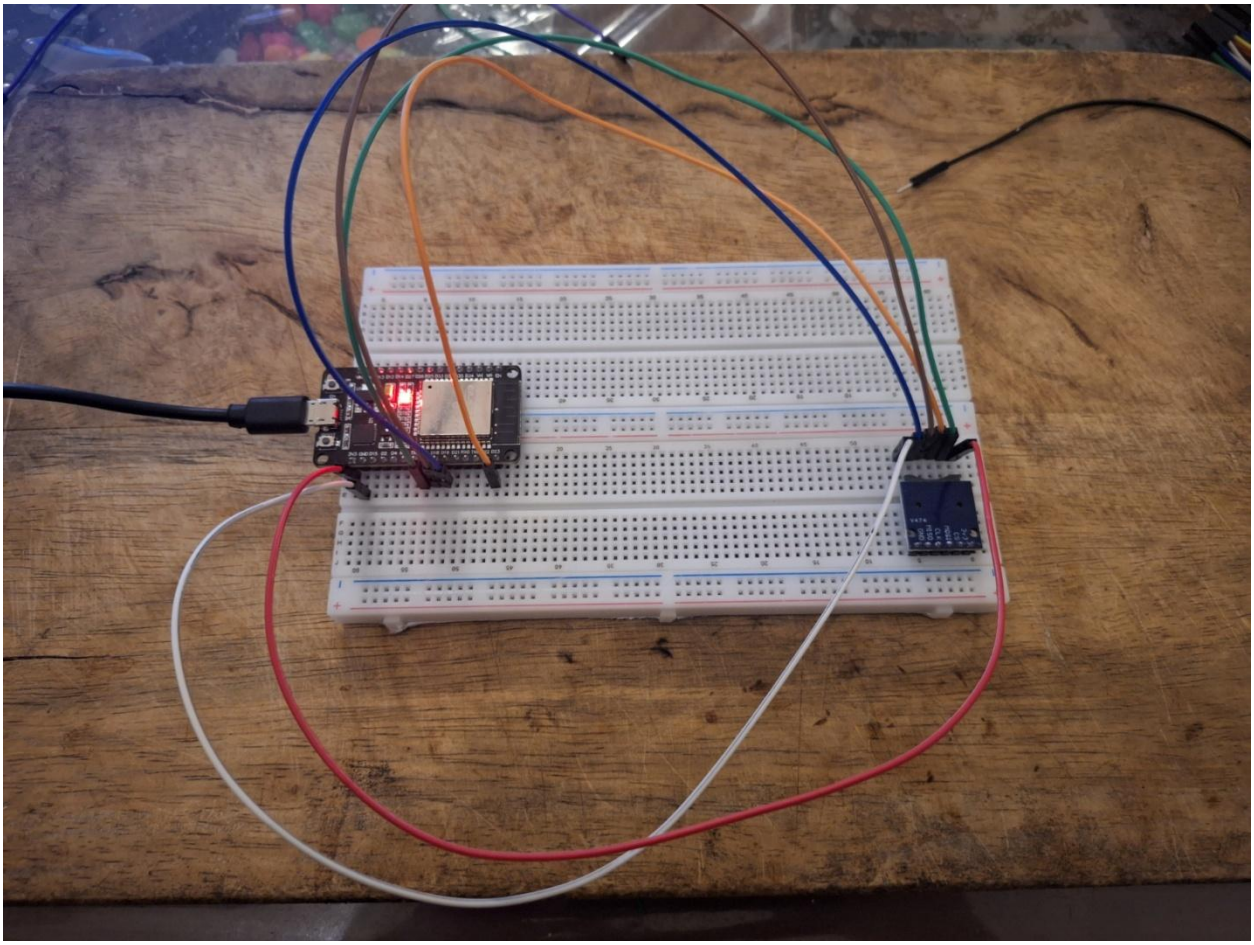
Verification: Verified by corresponding sequential console logs (Moving to 0 degrees, Moving to 90 degrees, Moving to 180 degrees) matching the physical motor positioning.



7: Micro SD Card Module Integration

Description: Physical breadboard wiring connecting a Micro SD card module (using SPI communication lines like MOSI, MISO, SCK, and CS) to the ESP32 development board.

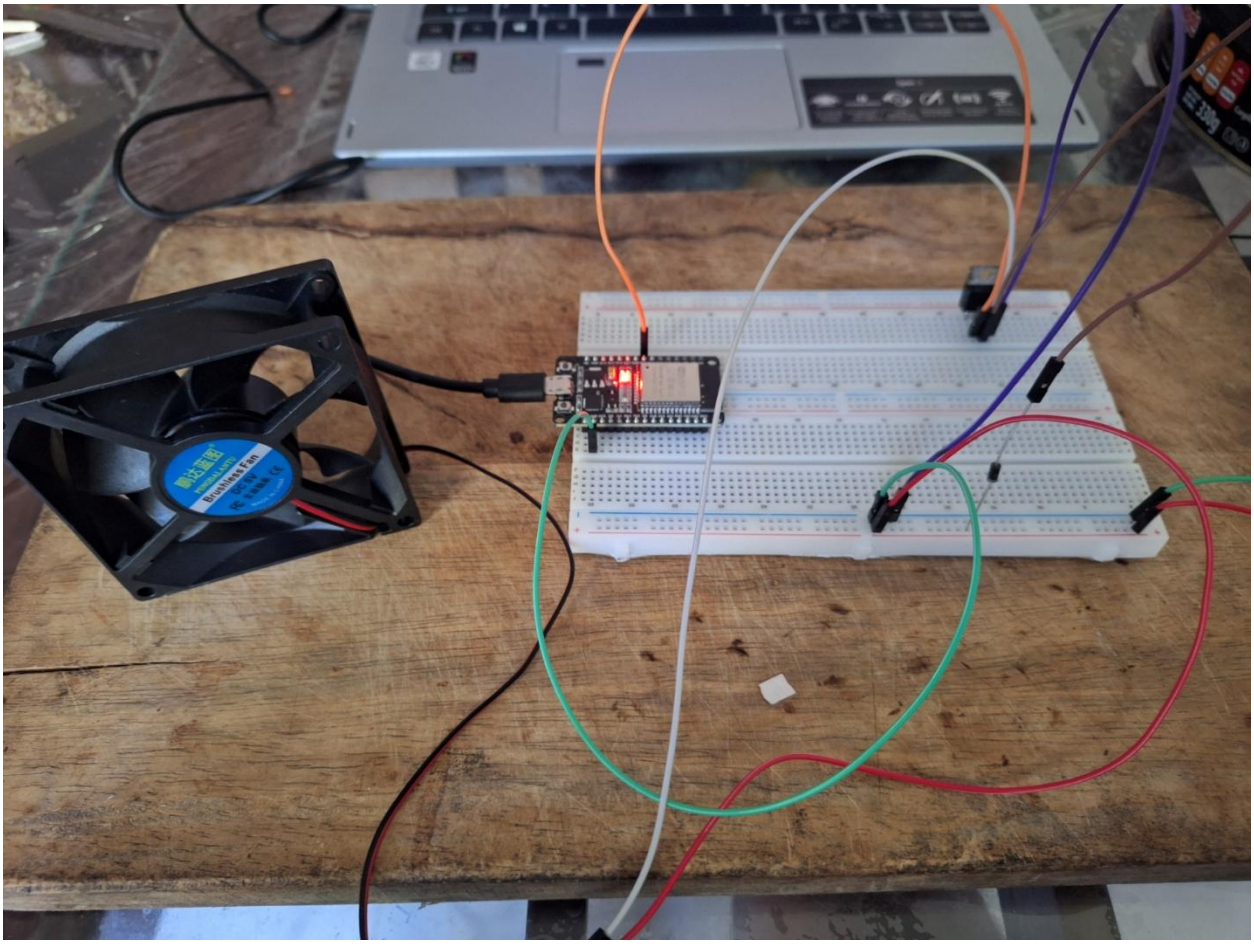
Verification: Confirmed by successful bus communication and initialization during testing.



8: DC Brushless Fan Hardware Integration

Description: Physical breadboard and jumper connection linking a DC brushless cooling fan to the ESP32 microcontroller digital output pin (fanPin = 25).

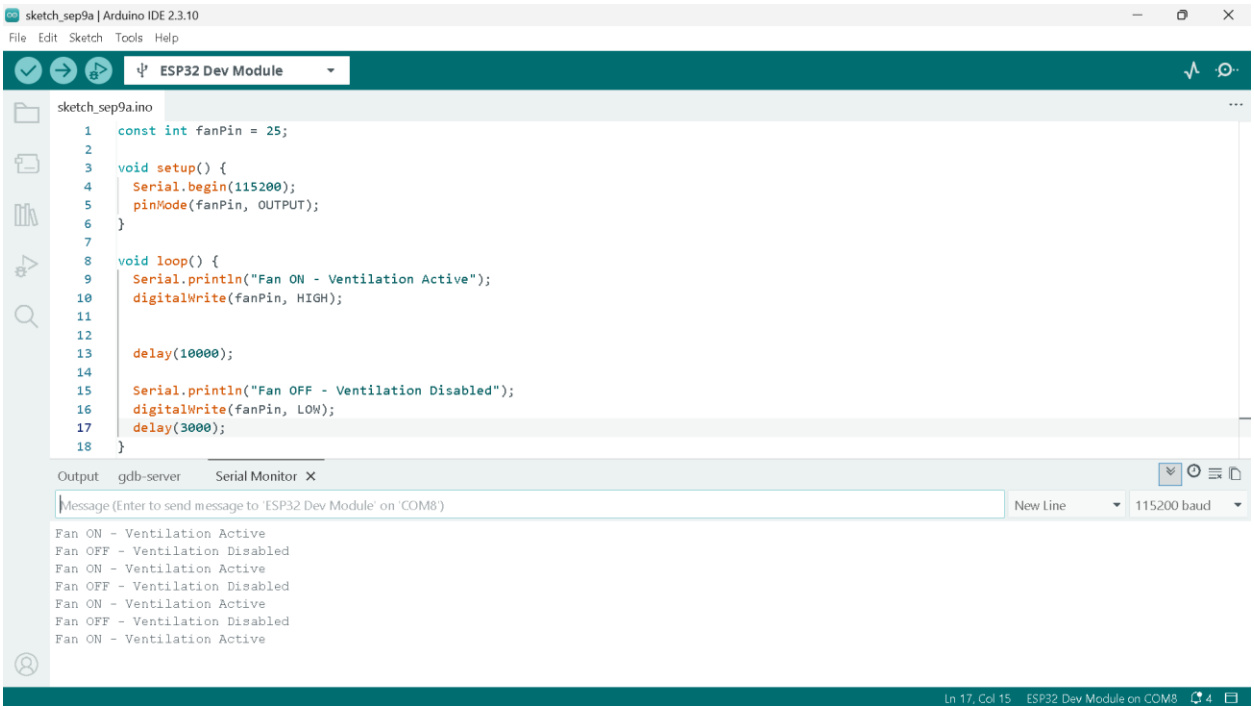
Verification: Confirmed by active mechanical rotation and air movement during execution.



Fan Control Firmware and Serial Output

Description: Arduino IDE code snippet configuring digital output control for ventilation management alongside status logging messages (Fan ON - Ventilation Active and Fan OFF - Ventilation Disabled).

Verification: Verified by active status updates streaming into the serial monitor output synchronizing with fan operation.



Next Steps

1: Physical Prototype Enclosure Construction

Description: Construct the outer enclosure and structural base of the prototype using rigiform to safely house the electronics and ensure rapid, easy modification.

2: Final Hardware Integration

Description: Integrate and secure all tested hardware components, including the ESP32 and sensors, within the designated, isolated sections of the rigiform base.

3: Wi-Fi Web Dashboard Finalization

Description: Complete the development of the web dashboard to allow users to remotely view live environmental data, weight changes, and system status alerts.